# Basics of LLMs: Speaker Script

Brown Bag session, 45 min, all levels. Presented by Sunil

## How to use this script

About 37 minutes of talking plus roughly 8 minutes of buffer for the audience's answers and questions. Read it aloud once beforehand. On the day, glance at the bold cues and talk in your own words rather than reading line by line.

| Slide | Title | Minutes | Clicks |
|---|---|---|---|
| 1 | Basics of LLMs | 1 | 0 |
| 2 | What is an LLM | 3 | 0 |
| 3 | Parameters | 2 | 4 |
| 4 | Tuning the numbers (radio) | 2 | 3 |
| 5 | Tokens | 3 | 5 |
| 6 | Inside the model | 4 | 5 |
| 7 | Prompts | 2 | 2 |
| 8 | Context window | 2 | 5 |
| 9 | Context rot | 3 | 5 |
| 10 | Using the window wisely | 2 | 4 |
| 11 | Embeddings | 3 | 5 |
| 12 | Cosine similarity | 3 | 3 |
| 13 | Pre-training | 3 | 5 |
| 14 | Post-training and close | 4 | 7 |

- **[Click 3]** means press the Right arrow a third time on that slide. The number is the step.
- **Ask the room** means stop and wait about 10 seconds. There are five real questions: slides 9, 11 and 13, and twice on slide 14.
- **If short on time** marks what to cut.
- Keys: Right arrow or Space next, Left arrow back, R replays a slide's animation, F fullscreen. The small circle at the bottom left lists every slide, so you can jump to any of them.

## Slide 1: Basics of LLMs (1 min)

Good morning everyone.

Today is Basics of LLMs. No code and no maths.

Parameters, tokens, context window, embeddings
we hear these words quite often.
Right now they may be jargon.
But In next 40 minutes, I will try to explain them.

Hi, I am Sunil and today we will try to understand What is an LLM,
what's inside it!
How does it reads text!
How it compares meanings!
and how it gets trained.

**Ask the room:**
Can anyone explain why it sometimes gives a confident but wrong answer.

## Slide 2: What is an LLM (3 min)

An LLM is a very powerful autocomplete.
We know the suggestions above our phone keyboard.
This is the same idea,
trained on far more text,
and it keeps going instead of stopping after one word.

You give it text.
It returns a probability for every possible next token.
For now, think of a token as roughly a word.

So we give a text to the model "The capital of India is".
The model does not look anything up. It scores every possible next piece.
"New" gets 86%, "Delhi" 7%, "the" 3%.
for simplicity lets say the model picks the most likely one, so "New" is added to the sentence.

Then the whole sentence goes in again, and "Delhi" gets 95%. Then a full stop.
Then it predicts an **end marker** and stops.
"New Delhi" was not retrieved from a database.
It was predicted piece by piece.

## Slide 3: Parameters (2 min)

We all have head “its a 70B parameter model”
or “its a 30B parameter model” and so on.
So what are parameters?
Just numbers. Numbers inside the model. ( knowledge of LLM)
This grid is a tiny representation of those parameters.
Initially they are random values, so the model knows nothing.

During training the model guesses the next token,
we compare it with the real one,
and then we nudge every number a little.

This cycle repeated trillions of times
Guess, compare, nudge
And with every cycle we try to drop the **error difference**.
The result is a good set of values, not a perfect one.

Every parameter is about **2 bytes**
so a 70B parameter model requires roughly 140 GB just to store it.
That is why big models run on GPU servers, hence the cost.

Sometimes people confuse Parameter with context.

Parameters are what the model learned.
They are fixed after training.
Context is what it is reading right now,
and it changes with every request.

## Slide 4: Tuning the numbers, the radio (2 min)

*Lets try to visualize this*
*we have a* old radio.
Consider each knob as one parameter.
Right now it is playing static, and the signal meter is low.

**[Click 1]** We turn one knob a little and it gets worse.
So we turn it back the other way. Better. So lets Keep it.
That is the whole rule:
**clearer, keep it; worse, turn it back.**

**[Click 2]** Now do that for every knob, many tiny turns.
we keep doing this until the signal is good enough (Not perfect)
and that is exactly how training ends too.

**[Click 3]** For a 70 billion parameter model there are 70 billion knobs (approx).
and nobody turns them by hand.
Training does the turning automatically.
gradients descent

## Slide 5: Tokens (3 min)

A token is the smallest unit an LLM processes.
A small program called a tokenizer
its a tool responsible for breaking
raw text down into pieces and
later converting the model's output back into human-readable text
Not whole words, not single letters, but chunks of words / sub-word

**[Click 1] [Click 2] [Click 3]**
https://platform.openai.com/tokenizer
Unbelievable
I am an AI Engineer

Why chunks/subwords? If the unit were whole words, the vocabulary would be huge and new words would break it. If it were letters, every text would become very long. Tokens are the middle road.

**[Click 4]**
one token is about three quarters of a word,
or about four characters.

**[Click 5]**
The Usage and cost of LLM APIs is typically measured in tokens, for both what goes in and what comes out.

**Optional**
**Step->**
**a) Text Normalization & Cleaning:**

Standardizes the input
removing excess whitespace, fixing formatting inconsistencies, and sometimes converting everything to lowercase depending on the model's design

**b) Splitting Text into Tokens**

**c) Mapping to Token IDs**
Every tokenizer has a fixed vocabulary
The tokenizer takes the split tokens
and assigns each one its corresponding Numerical ID from that vocabulary.
These IDs are what get passed into the LLM

**Out-of-Vocabulary (OOV) word.**

*__Fallback to a Special Token(old tokenizers)__ - [UNK] or <unk>
*__Subword Decomposition (modern)__ - rarely output [UNK] because they break unknown words down into smaller subword
*__Byte-Level Fallback__

## Slide 6: Inside the model (4 min)

So where do those 70 billion numbers actually sit?
Inside a design called the **transformer**.
It has four parts, and every part is made of parameters.
We will use the same sentence, “The capital of India is”.

**[Click 1] Embedding.** Tokens become numbers. Each token is looked up as a vector, which is just a list of numbers. This is about 2% of the parameters.

**[Click 2] Attention.** Which tokens should I look at? For the word “is”, the model looks mostly at “India” and “capital”. The thicker the line, the more attention. It is recomputed for every input. About 17% of the parameters.

**[Click 3] Feed-forward.** This is where it looks up patterns. It recognizes what the token means here and adds what the model learned in training. Picture it stretching wide, units firing, and squeezing back, one token at a time. Researchers believe this part stores much of what the model learned, and it holds about 80% of the parameters. Notice the dashed box: attention plus feed-forward is one block, repeated around 80 times in a model this size.

**[Click 4] Output.** The result becomes scores, then probabilities, for every possible next token. “New” 86%, “Delhi” 7%. That is the exact table we saw on slide 2. About 2% of the parameters.

**[Click 5]** So where do the 70 billion sit? About 80% in feed-forward, about 17% in attention. This is approximate, for a Llama-style 70B model, and the split varies by model.

If you remember one line from this slide: text goes in as numbers, probabilities come out, and most of the model is the part that holds what it learned.

*If short on time: do clicks 1 to 4 in one breath each and skip the percentages on click 5.*

## Slide 7: Prompts (2 min)

What is a prompt?
A prompt is everything the model reads before it starts predicting.
Not just your question.
Right now we see only the user part, the thing the person types.

**[Click 1]** But a developer puts a system prompt in front of it: the role, the rules, the tone, the format. Here it says “You are a helpful assistant. Keep answers short.” The end user usually never sees it. Look at the token count at the top jump.

**[Click 2]** To the model, all of this is one long sequence of tokens, with special markers showing who is speaking. There is no hidden second channel. It reads the lot and predicts the next token, same as before.

## Slide 8: Context window (2 min)

Context window is the most tokens a model can handle in one request.
Your input and the model’s reply share it.
Models today are roughly 200K to 1M tokens, and some are larger.
One million tokens is roughly 750,000 words, or 1,500 to 3,000 pages.(Rough approximation)

Here is an example 200K window. Empty. **[Click 1]** First the system prompt goes in, a thin sliver. **[Click 2]** Then the conversation history. Each block is one turn, and the whole history is sent again with every message, so this part grows fast. **[Click 3]** Attached files and tool results, such as web search or database output, take space too. **[Click 4]** And the model’s own reply counts as well.

**[Click 5]** Keep chatting and the window fills up. When it is full, one of two things happens. The request fails with an error, or the app quietly drops or summarizes the oldest content. That second one is why a long chat can suddenly seem to forget how it started.

tool definitions, images and the “thinking” tokens of reasoning models count as well.

## Slide 9: Context rot (3 min)

Quick question for the room.
Our context window can fits 10 documents, and only 2 are relevant to the question.
Do you send all 10, or just the 2?

**[Click 1]** Send just the 2. It fits, but that does not mean it works well.

**[Click 2]**
Three reasons.
**Quality**: more noise for the model to sort through.
**Cost**: input tokens are typically billed on every request.
**Speed**: more tokens take longer to process.
On the right, all 10 is about 80K tokens, just 2 is about 16K.

**[Click 3]** This lets to context rot.
The more tokens in the context,
the less reliably the model uses any of them, even when everything fits.

**[Click 4]** What it looks like in practice.
Facts that are in the context get missed.
Answers get less accurate.
Long chats drift off course.
Example
you ask “What deadline did I mention at the start?”
and the model says
“I don’t see a deadline in our conversation.”

**[Click 5]**
Chroma Research tested 18 models in 2025, and all of them got less reliable as input grew.
(**Skip mostly**)Even a single distractor lowered accuracy. On one memory test, a focused prompt of about 300 tokens beat the full version of about 113K tokens.

So a bigger window is not a license to fill it.

## Slide 10: Using the window wisely (2 min)

So the generic advice is:
don't aim to fill the window. Quality fades gradually, so aim for the smallest set of high-signal tokens that gets the job done. That phrasing comes from Anthropic's engineering write-up, which is listed at the bottom.

**[Click 1]**
A rule of thumb:
start wrapping up when the chat is around 40 to 70% full.
**(Disclaimer)**Please note, there is no official number.
It varies by model and task, so treat it as a guide.

**[Click 2]**
Wrap up by handing off.
Ask the model to write a note: the goal, decisions made, current state, next steps, and key files.

**[Click 3]**
Paste that note into a new chat and carry on.
Look at the new chat: about 6% full, and the model is reliable again.

**[Click 4]** Three habits.
One chat per topic.
Send only relevant files.
Drop stale tool output.

## Slide 11: Embeddings (3 min)

We have talked about text, tokens and numbers.
Now the big question:
how does a model hold meaning?
The answer is an embedding.
An embedding is a vector that stands for the meaning of text.
A vector is just a list of numbers.
Similar meaning gives nearby vectors.
directions between vectors carry meaning too.

On the map, you can see king, queen and man. Notice the arrow from king to queen.

**[Click 1] Ask the room:** where would woman land?
**[Click 2]**
**[Click 3]**

**[Click 4]** The takeaway.
No single number in a vector means anything on its own.

The meaning is in the whole vector, in distances and in directions.
words with similar meaning end up near each other,
and groups like animals or cities form clusters.

**[Click 5]** A common confusion, so let me clear it up.
When the tokenizer processes the word,
it gives each token one ID number, for example queen is 31052.
That is only a label. It carries no meaning.
The embedding turns that label into a vector, and the meaning lives there.

A small **disclaimer**, also written at the bottom of the slide: this is the classic textbook picture. Real embeddings have hundreds or thousands of dimensions, and the axes have no labels.

## Slide 12: Cosine similarity (3 min)

**question**
How do we tell if two vectors are close?
Let me show why that matters with an example.
A user asks “How do I get my money back?”
We have two paragraphs.
A is a returns policy about refunds.
B is cafeteria timings.

**[Click 1]**
The obvious idea: just match the words.
word matching finds nothing, yet paragraph A answers the question perfectly.

**[Click 2]**
So we turn both into vectors.
Texts with similar meaning give similar vectors.

**[Click 3]** To score that, we use cosine similarity.
It compares the direction of two vectors.
Near 1 means matching.
Near 0 means not matching.
No formula needed: think of it as asking whether two arrows point the same way.

Takeaway: we compare meaning by direction, not by shared words.

## Slide 13: Pre-training (3 min)

We now know what is inside the model and how it handles text. Last topic: how did we get from random numbers to something that chats with you? Two stages. Stage one is pre-training. In one line: guess the next token, trillions of times.

**[Click 1]** The data: a large share of the public internet, plus books and code. Trillions of tokens of text.

**[Click 2]** The loop: show the start of a text, hide the next token, compare, nudge. It is the same loop from the parameters slide. For example: “The capital of France is”, and the real next token is “Paris”. If the guess was off, nudge the parameters.

**[Click 3]** The cost: thousands of GPUs, running for months. Managers, this is why only a small number of organisations train these models from scratch.

**[Click 4] Ask the room:** the result is called a base model. It knows the language and a lot of facts. If I type “What is the capital of Germany?”, what will it reply? Take two or three guesses.

**[Click 5]** It continues the pattern. It replies “What is the capital of Italy? What is the capital of Spain?” It was trained to continue text, not to answer questions. Like an autocomplete that finishes your exam paper with more exam questions. So how do we turn it into an assistant? Next slide.

## Slide 14: Post-training and the close (4 min)

*On screen: label “How an LLM is trained, 2 of 2”. Seven clicks: step 1, a question, a problem, step 2, a question, a problem, and the summary.*

Stage two is post-training: from a text continuer to an assistant.

**[Click 1]** Step one: we show it thousands of questions together with ideal answers. For example, question “What is the capital of Germany?”, ideal answer “The capital of Germany is Berlin.” It learns: a question gets an answer. This is called supervised fine-tuning.

**[Click 2] Ask the room:** what problem could this create? Give it 15 seconds and let people answer.

**[Click 3]** Hallucination. It answers confidently even when it does not know. Look at the example: we ask who wrote a book called “The Silent Lantern of Oslo”. That book does not exist, and the model names an author anyway. It is like a student who fills the exam sheet instead of leaving it blank. Every training example had an answer, and none said “I don’t know”.

**[Click 4]** Step two: show it two answers and tell it which one humans prefer. Repeat that millions of times. Here, answer A is a confident author name, answer B is “I can’t find reliable information about that book”. This is reinforcement learning from human feedback.

**[Click 5] Ask the room:** what could be the issue with this step? Again, 15 seconds.

**[Click 6]** Human bias. People like confident, flattering and longer answers, so models learn to tell us what we want to hear.

**[Click 7]** Every organisation does its own post-training, so models get different personalities. Pre-training gives knowledge. Post-training gives behaviour.

Thank you. Questions?

Questions you may get, with short answers

Keep each answer to about 20 seconds. If you do not know, say “Good question, I will check and come back to you.” That is better than guessing.

## About predicting the next token

**Q: Does it always choose the highest-probability token?**

No. Always taking the top one is called greedy decoding, and it gives dull, repetitive text. Most products sample from the probabilities instead, so a token with 7% chance is picked about 7% of the time. A setting called temperature controls this: low means close to always the top token, high means more variety. Related settings such as top-p and top-k cut off the very unlikely tail. On slide 2 I picked the top one only to keep the example simple.

**Q: Why does the same question give different answers?**

The sampling above. Even at the lowest temperature, results are not always identical, because of how the computation is run on the hardware. For testing, check properties of the answer, not exact text.

**Q: Does it actually understand, or is it just predicting?**

It is trained only to predict the next token. To do that well it learns a lot of structure about language and the world, and it often behaves as if it understands. Whether that counts as real understanding is still debated, so I would not claim either way. What matters at work: it can be right and wrong in the same confident tone.

**Q: Why does it hallucinate, and can we stop it?**

It produces the most plausible continuation, and plausible is not the same as true. It also cannot reliably tell what it does not know. We cannot fully stop it. We reduce it by putting the facts in the context, asking for sources, and checking the output.

**Q: Can it browse the internet or look things up?**

The model alone cannot. An app around it can add tools such as web search or a database. Their results are placed in the context window, which is the “tool results” block on slide 8.

**Q: Why does it get simple things like counting letters or arithmetic wrong?**

It sees tokens, not letters, so counting the letters in a word is hard for it. Arithmetic is pattern prediction, not calculation. Products fix this by letting the model call a calculator or run code.

## About training and size

**Q: Does it learn from my chats?**

Not while you chat. The parameters are fixed after training. A “memory” feature simply saves notes and puts them back into the context next time. Separately, whether a provider uses your data for future training depends on its policy and your plan, so check your company’s setup.

**Q: Why does it not know recent events?**

Its training data stops at a date, called the knowledge cutoff. Anything later it can only know if it is given the information in the context or fetched by a search tool.

**Q: What is the difference between training and using the model?**

Training sets the parameters and takes thousands of GPUs and months. Using the model, called inference, only reads the parameters and predicts tokens. Training happens once, and using it happens millions of times.

**Q: Is a bigger model always better?**

Usually more capable, but not always. Bigger models cost more and run slower. The quality of the training data and of post-training matters a lot too. For many simple tasks, a smaller model is enough.

**Q: What is fine-tuning, and how is it different from prompting?**

Prompting changes only the input, and nothing in the model changes. Fine-tuning does extra training that changes the parameters. Try better prompting and better context first, because it is cheaper and faster.

**Q: What is a reasoning model?**

A model that writes extra “thinking” tokens before it gives the final answer. Those tokens take space in the context window and cost money, which is the line at the bottom of slide 8.

## About tokens, context and embeddings

**Q: How many tokens is a page of text?**

Using the rule of thumb on slide 5, a page of about 500 words is roughly 650 to 700 tokens. It is only an estimate, and it varies by model and language.

**Q: What is the difference between the context window and memory?**

The context window is everything the model reads in this one request. Memory features are separate: they store notes outside the model and paste them into the context when needed. Anything in memory still uses space in the window.

**Q: If windows keep getting bigger, is context rot solved?**

No. A bigger window lets more fit, but the research on slide 9 shows reliability can still drop as the input grows. Large inputs also cost more and take longer.

**Q: Are the embeddings on slide 6 the same as the ones used for search?**

The idea is the same, but they are separate things. The embedding layer inside an LLM is part of that model. For search and matching, applications usually use a separate, smaller embedding model that is built to turn a whole text into one vector.

**Q: What does "dimensions" mean?**

How many numbers are in the vector. Our map has 2 so we can draw it. Real embeddings have hundreds or thousands.

**Q: Why cosine similarity and not just distance?**

Cosine compares direction and ignores the length of the vectors, and direction is what carries meaning here. Other measures exist and are also used. For this talk, "do the arrows point the same way" is enough.

**Q: How does this connect to RAG?**

RAG means finding the most relevant documents using embeddings and cosine similarity, and putting only those into the context. It combines slides 9 to 12, and it is a good topic for a follow-up session.

## Practical and company questions

**Q: What is the difference between an LLM and ChatGPT or Claude?**

The LLM is the model, the parameters that predict tokens. The product around it adds a system prompt, tools, memory and the chat screen. That is why two products on the same model can behave differently.

**Q: Can we run a model ourselves, inside the company?**

Yes, with open-weight models, but you need GPU servers sized to the model (slide 3: 70B is about 140 GB at that precision). Smaller or compressed versions need less. It is a trade-off between control, cost and quality.

**Q: Is it safe to paste company or customer data into an AI tool?**

It depends on the tool, the plan and the company policy. Follow your organisation's approved tools and data rules, and when in doubt, do not paste it. Ask the security or compliance team.

**Q: How do we test an application built on an LLM?**

Check properties instead of exact text: is it correct, on topic, in the right format, safe. Run the same input several times, use a fixed set of example questions, and re-run it whenever the model, prompt or context changes. Track the results over time.

**Q: Will it replace developers or testers?**

Nobody knows for sure. It speeds up routine work, but its output still needs checking, and the ability to verify and test it matters more, not less.

**Q: Where should I start using it at work?**

Pick small, low-risk tasks where you can check the answer yourself: summaries, drafts, first versions of code or tests. Give it good context, and always review the output.